Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 2 – Project Management Plan

– Ho Chi Minh City, October 2026 –

Table of Contents

I. Record of Changes 3

II. Project Management Plan 4

1. Overview 4

1.1 Scope & Estimation 4

1.2 Project Objectives 5

1.3 Project Risks 7

2. Management Approach 8

2.1 Project Process 8

2.2 Quality Management 9

2.3 Training Plan 10

3. Project Deliverables 10

4. Responsibility Assignments 14

5. Project Communications 15

6. Configuration Management 15

6.1 Document Management 15

6.2 Source Code Management 15

6.3 Tools & Infrastructures 16

I. Record of Changes

Date	A* M, D	In charge	Change Description
04/08/2026	A	Nguyễn Phương Mỹ Thuận	Initial draft
25/10/2026	M	Nguyễn Phương Mỹ Thuận	Aligned with the implemented system for Review 2: WBS item names (Lazada + AURELLE instead of Lazada + Tiki, BRKGA + CP-SAT packing, warehouse, GHN shipping and returns; mobile app kept as Iteration 3 plan), non-functional requirements with metric, threshold and measurement method, test figures, risks, training plan, source-code management (actual branches, commit convention, CI), tools and infrastructure (Docker, Kubernetes, GitHub Actions; AWS / DigitalOcean removed). Effort estimates are unchanged.
08/11/2026	M	Nguyễn Phương Mỹ Thuận	Test status updated to 08/11/2026; diagram tools changed from PlantUML / Mermaid to draw.io.

*A - Added M - Modified D - Deleted

II. Project Management Plan

1. Overview

1.1 Scope & Estimation

Complexity classification criteria used in the WBS table below: Simple = a self-contained task with no third-party API dependency, estimated at ≤ 5 man-days; Medium = involves integrating one external API/module or requires moderately complex data design, 5–8 man-days; Complex = involves an algorithm/optimization problem (3D bin packing), integrates multiple third parties simultaneously, or affects multiple other modules, > 8 man-days. Effort figures are the original estimates; item names were updated on 25/10/2026 to the scope actually implemented.

#	WBS Item	Complexity	Est. Effort (man-days)
1	**Package 1: Architecture, Core Setup & Security**		**24**
1.1	Design system architecture & database schema (MongoDB)	Simple	7
1.2	Build base back end (NestJS), REST API & security (JWT, RBAC, two-factor)	Medium	8
1.3	Build the scheduler (@nestjs/schedule) for periodic jobs (order sync, packing-plan computation, status write-back)	Simple	3
1.4	Set up base front end (React + Vite staff portal) and the AURELLE web store (Next.js)	Medium	6
2	**Package 2: Omnichannel Integration Module**		**26**
2.1	Integrate Lazada Open Platform (OAuth, order and catalog sync, masked recipient data)	Complex	8
2.2	Integrate AURELLE Open API (OAuth, HMAC-signed webhooks, order / return status and stock write-back)	Complex	8
2.3	Polling mechanism to synchronize order-status updates (update_after watermark, paging)	Simple	6
2.4	Normalize omnichannel order data into the internal order model	Simple	4
3	**Package 3: Packing Optimization Module**		**33**
3.1	Order consolidation (order groups, link, split, merge)	Medium	7
3.2	3D bin-packing engine: BRKGA heuristic with EMS decoder (TypeScript) and CP-SAT proof service (Python, OR-Tools)	Complex	16

#	WBS Item	Complexity	Est. Effort (man-days)
3.3	Cost calculation (packaging materials, shipping fee quote)	Medium	6
3.4	UI screen for Packaging Staff to review the 3D plan	Simple	4
4	**Package 4: Fulfillment Management Module**		**26**
4.1	Warehouse bins, FIFO stock lots, stock reservation and picking list	Medium	5
4.2	Mobile app to scan barcodes / QR and update picking / packing status (planned for Iteration 3; web-portal scanning implemented first)	Complex	10
4.3	PDF generation for packing slip, shipping label, manifest and warehouse labels	Medium	6
4.4	Shipping via GHN, delivery tracking and returns	Medium	5
5	**Package 5: Analytics and Reporting**		**15**
5.1	Packing reports and home dashboard	Medium	6
5.2	Packaging-cost and packaging-material savings analysis	Medium	5
5.3	Action history and CSV export	Simple	4
	Total Estimated Effort (man-days)		**124**

Package 3 (Packing Optimization) accounts for 33 man-days — the largest share of effort among the five packages (~27% of total effort) — because it is the only package that solves an algorithmic optimization problem (3D bin packing is NP-hard) rather than standard CRUD / API integration work. This is also why risk R02 (section 1.3) is rated High Impact.

Work implemented beyond the original WBS item names (warehouse structure and internal SKUs, returns and exchanges, AURELLE web store) is included in items 1.4, 2.2, 4.1 and 4.4 above.

1.2 Project Objectives

The project's core objective is to develop an MVP (Minimum Viable Product) of the AOFP system (OptiPack AI) within one semester (~15 weeks, matching the schedule in section 3). The system must synchronize orders from Lazada and the AURELLE web store, consolidate orders of the same recipient, guide picking, compute 3D packing plans with a bin-packing engine, book shipments with GHN and handle returns. The goal is an internal tool that digitizes the warehouse process and reduces packaging and shipping costs. A mobile app for scanning is planned for Iteration 3.

Non-functional Requirements (quantified):

ID	Quality attribute	Metric	Threshold	Measurement method
NFR-1	Performance	Time from a new Lazada order to its appearance in OptiPack	≤ configured sync interval + 1 minute (default ≤ 6 minutes)	Create a test order on the Lazada shop and compare its creation time with the sync job log
NFR-2	Performance	Packing-plan computation time per order (BRKGA engine)	p95 ≤ 1 second on the 300-order benchmark (last run: p95 560 ms, be/scripts/benchmark-results/solver-v2.json); CP-SAT proof limited to 6 seconds per order	scripts/solver-benchmark.ts and scripts/cpsat-benchmark.ts (back end)
NFR-3	Scalability	Orders imported and grouped per hour	≥ 500 orders / hour	Load test with generated orders during the Verification stage (target; not yet executed)
NFR-4	Reliability	Status changes written back to AURELLE	0 lost changes; retried up to 8 times with back-off 30 s → 30 min	Stop the AURELLE simulator, make changes, restart, verify delivery (outbox)
NFR-5	Availability	Packing plans when the CP-SAT service is down	Plans still produced (label "heuristic")	Stop the packer service and compute a plan
NFR-6	Security	Failed logins before lockout; role checks	Lock after 5 failures for 15 minutes; an unauthorized role receives HTTP 403	Automated tests (Jest) on authentication and role guards

#	Testing Stage	Test Coverage	No. of Defects	% of Defect	Notes
1	Reviewing		< 15	N/A	Cross-review of source code through Pull Requests and of UI/UX design.
2	Unit Test	> 75% (target)	< 10	< 5%	Focus on the packing engine (BRKGA, validator), order consolidation, stock reservation and picking logic.
3	Integration Test	> 80% (target)	< 10	< 5%	Lazada and AURELLE synchronization, webhooks and write-back (AURELLE simulator and conformance script), GHN staging API.
4	System Test	100% Core Flow	< 5	< 2%	End-to-end flow on the web portal: order intake → picking → packing →

#	Testing Stage	Test Coverage	No. of Defects	% of Defect	Notes
					shipment → delivery / return.
5	Acceptance Test	100%	0 (Critical)	0%	Acceptance run-through with the supervisor at the capstone defense.

Milestone Timeliness (%): 100% of product delivery milestones (Packages) completed on schedule according to the Sprint timeline (2 weeks / Sprint).

Allocated Effort (man-days): total estimated effort is 124 man-days (WBS table), allocated across phases as follows:

- Requirement & Analysis: 15 man-days (~12%)
- Design (System, Database, UI/UX): 19 man-days (~15%)
- Coding (Web, Mobile, packing engine, API): 63 man-days (~51%)
- Testing (Unit, Integration, E2E): 20 man-days (~16%)
- Project Management & Docs: 7 man-days (~6%)

Note: the "No. of Defects" figures are calculated against an initial estimate of roughly 120 test cases for the whole system. Status as of 08/11/2026: the back end has more than 1,490 automated Jest tests passing (150 spec files) and the packer service has pytest tests; all run in GitHub Actions on every push. The functional test-case total for the whole system (including manual / UAT testing) is finalized in Report 5.

1.3 Project Risks

#	Risk Description	Impact	Possibility	Response Plans
R01	Rate limits, API changes or refused developer access from sales channels and the carrier (Lazada, GHN) — a risk already encountered with Shopee and TikTok Shop, which led the team to Lazada and a team-built second channel (AURELLE).	High	Medium	Adapter layer per channel (LazadaAdapter, AurelleAdapter sharing one protocol client) and per carrier (GhnAdapter, mock adapter); retries with exponential back-off on API calls; idempotent webhook processing; AURELLE conformance script and simulator.
R02	Packing engine too slow or poor quality on large orders.	High	Medium	Deterministic search budget (number of placement checks) instead of wall-clock time; built-in BRKGA always available, CP-SAT optional with a 6-second limit; independent validator; benchmark scripts compare solver versions.
R03	Difficulty developing the mobile app (Flutter): camera scanning and printer connection need native-channel experience.	High	Medium	Scanning is implemented first on the web portal (barcode scanner / manual entry); the mobile app is planned for Iteration 3 with dedicated Flutter training; prefer existing libraries (mobile_scanner, blue_thermal_printer).

#	Risk Description	Impact	Possibility	Response Plans
R04	Lack of realistic test data: Lazada has no sandbox (the Loan Test Account cannot be connected through Open API) and masks recipient data; GHN staging does not perform real pickup.	Medium	Medium	Test the real API with the team's KYC-verified Lazada shop; AURELLE web store and simulator generate full test orders; GHN staging plus manual status buttons for demo; seed scripts for demo data.
R05	Schedule slippage due to the heavy back-end workload (multi-channel synchronization, packing engine, warehouse, shipping, returns).	High	Medium	Implement order synchronization and the highest-risk module (packing) first; review actual effort every Sprint and adjust early.
R06	Inaccurate packing recommendations leading to wrong boxes.	High	Medium	Packaging Staff must approve every plan before packing; parcels whose measured weight differs from the estimate by more than the tolerance are held for review by another person; every adjustment and rejection is logged and summarized in the packing feedback report.
R07	Leaked API keys / secrets or exposure of customer data (name, phone, address).	High	Low	Secrets in environment variables (.env, not committed); marketplace tokens encrypted with AES-256-GCM; the AI text service receives no customer data; sync only the fields needed for fulfillment.
R08	No staging environment for the Review 2 demo.	High	Medium	Deploy the existing Docker Compose / Kubernetes setup to a hosted staging environment and seed data with the repository scripts.

2. Management Approach

The project is managed with the Agile/Scrum framework. This approach suits a software project that integrates optimization algorithms and must adapt to changes in the APIs of sales channels and carriers.

Throughout the project the team uses Jira to track Sprints, Backlog and Issues (ticket ids AOFP-<number> are used in every commit), Confluence and the repository folders GUIDE_DOC and docs to store documentation, and GitHub for source code management.

2.1 Project Process

A. Initiation phase (Sprint 0): set up the project environment, train the team on the collaboration tools (Jira, Confluence, GitHub), scaffold the base architecture (back end, staff portal, AURELLE web store) and build the initial Product Backlog from the Major Features of Report 1.

B. Sprint Execution phase: the project is divided into two-week Sprints. Each Sprint includes the following ceremonies:

1. Sprint Planning: select User Stories from the Product Backlog into the Sprint Backlog and assign tasks (database design, back-end API, front-end UI…).
2. Daily Scrum: members update daily progress and flag blockers with the algorithm or integrations.
3. Development & Testing: code is pushed to GitHub; Pull Requests are cross-reviewed by another member and must pass CI before being merged.
4. Sprint Review: completed features are demoed to the supervisor at the end of the Sprint.
5. Sprint Retrospective: internal meeting on lessons learned.

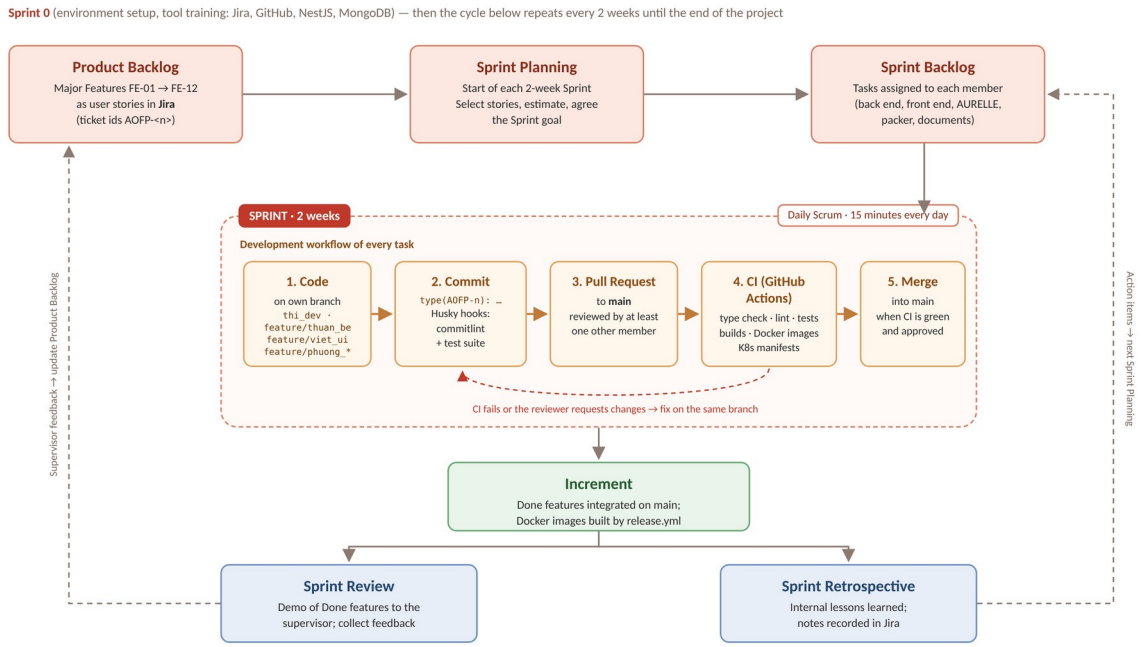


Figure 1. Scrum process and development workflow applied by the team

C. Release & Deployment phase

- Perform System Testing and Acceptance Testing across the entire system flow.
- Package the MVP, build Docker images, write the user guide and prepare for the Capstone Project Defense.

2.2 Quality Management

- Defect Prevention: coding conventions per language (TypeScript / NestJS for the back end, React and Next.js for the front ends, Python for the packer service). Static analysis runs automatically: ESLint (strict type-checked rules) and TypeScript type checking for back end and front ends, ruff and mypy for the packer service.
- Reviewing: Pull Request workflow on GitHub; changes are merged into main only through Pull Requests that pass CI and are reviewed by at least one other member. If two reviewers disagree on a PR, the Leader (Thuận) makes the final call.
- Unit Testing: the back-end developers (Thuận, Thi) write Jest unit tests for the most complex functions — packing engine and validator, order consolidation, stock reservation, picking and returns. A Git pre-commit hook runs the test suite and a commit-message hook checks the commit convention.
- Integration Testing: AURELLE integration is checked with the conformance script and simulator; GHN with smoke-test scripts on the staging API; the Lazada sync with the real shop, verifying that the update_after watermark does not skip records.

- System Testing: end-to-end testing across the full flow — order from the sales channel → consolidation and picking → packing plan and packing session → shipment and delivery / return. Defects are logged and tracked in Jira.

2.3 Training Plan

Training Area	Participants	When, Duration	Waiver Criteria
Project management workflow (Jira, Confluence, GitHub)	Whole team	1 week	Mandatory
3D bin-packing optimization (BRKGA, Empty Maximal Spaces, OR-Tools CP-SAT)	Thuận, Thi (Back end)	Sprint 1 (2 weeks)	Mandatory
Web front end (React, Next.js) and mobile app development (Flutter, for Iteration 3)	Việt, Phượng (Front end / Mobile)	Sprint 1 (2 weeks)	Mandatory
Lazada Open Platform, AURELLE Open API and GHN API integration (OAuth, request signing, webhooks)	Thuận, Thi (Back end)	1 week	Mandatory

3. Project Deliverables

Below is the list of the project's main deliverables, arranged according to the development lifecycle (Agile/Scrum). Due dates are estimated from Sprint end dates.

#	Deliverable	Duration	Start	Finish	Resource Names	Week	Notes
1	Project AOFP	65 days	07/09/26 08:00	18/12/26 17:00			
2	Project Start	0 days	07/09/26 08:00	07/09/26 08:00		Week 1	
3	Initiation (Stage 1)	5 days	07/09/26 08:00	11/09/26 17:00		Week 1	
4	Prepare Report 1	5 days	07/09/26 08:00	11/09/26 17:00	Whole team	Week 1	
5	Deliver Report 1 (Project Introduction)	0 days	11/09/26 17:00	11/09/26 17:00	Thuận	Week 1	
6	Planning & Requirement (Stage 2)	10 days	14/09/26 08:00	25/09/26 17:00		Week 2-3	
7	Project Planning	5 days	14/09/26 08:00	18/09/26 17:00		Week 2	
8	Prepare Project Plan v0.9	5 days	14/09/26 08:00	18/09/26 17:00	Thuận	Week 2	

#	Deliverable	Duration	Start	Finish	Resource Names	Week	Notes
9	Deliver Report 2 (Project Plan v0.9)	0 days	18/09/26 17:00	18/09/26 17:00	Thuận	Week 2	
10	Project Preparing	10 days	14/09/26 08:00	25/09/26 17:00		Week 2-3	
11	Create Technical Prototype	10 days	14/09/26 08:00	25/09/26 17:00	Thi, Việt	Week 2-3	Scaffold the base NestJS / React / Next.js architecture
12	Technical Training / Self Study	10 days	14/09/26 08:00	25/09/26 17:00	Whole team	Week 2-3	Training on NestJS, MongoDB, bin-packing algorithm design
13	Requirement Analyzing	10 days	14/09/26 08:00	25/09/26 17:00		Week 2-3	
14	Write SRS v0.9	10 days	14/09/26 08:00	25/09/26 17:00	Thuận, Thi	Week 2-3	
15	Update Project Plan (v1.0)	2 days	23/09/26 08:00	25/09/26 17:00	Thuận	Week 3	
16	Deliver Report 3 (SRS v0.9 + Plan v1.0)	0 days	25/09/26 17:00	25/09/26 17:00	Thuận	Week 3	
17	Software Designing (Stage 3)	10 days	28/09/26 08:00	09/10/26 17:00		Week 4-5	
18	Write SDS (System Design Specification)	10 days	28/09/26 08:00	09/10/26 17:00	Thuận, Thi	Week 4-5	Design the database schema (MongoDB) & system architecture
19	Update Project Plan (v1.1)	2 days	07/10/26 08:00	09/10/26 17:00	Thuận	Week 5	
20	Deliver Report 4 (SDS v0.9 + Plan v1.1)	0 days	09/10/26 17:00	09/10/26 17:00	Thuận	Week 5	
21	Implementation (Stage 4)	30 days	12/10/26 08:00	20/11/26 17:00		Week 6-11	
22	Code Iteration 1 (Packages 1 & 2: Core & Omnichannel)	10 days	12/10/26 08:00	23/10/26 17:00		Week 6-7	Integrate Lazada and AURELLE + begin Fulfillment (Package 4)
23	Update SRS & SDS (Iteration 1)	5 days	12/10/26 08:00	16/10/26 17:00	Thuận	Week 6	

#	Deliverable	Duration	Start	Finish	Resource Names	Week	Notes
24	Create Test Cases (Iteration 1)	5 days	14/10/26 08:00	20/10/26 17:00	Thi, Việt	Week 6-7	
25	Code, UT, IT (Iteration 1)	8 days	14/10/26 08:00	23/10/26 17:00	Thi, Việt, Phượng	Week 6-7	
26	Complete Software Package 1 & 2	0 days	23/10/26 17:00	23/10/26 17:00	Whole team	Week 7	
27	Code Iteration 2 (Package 3: Packing Optimization)	10 days	26/10/26 08:00	06/11/26 17:00		Week 8-9	BRKGA packing engine and CP-SAT proof service
28	Update SRS & SDS (Iteration 2)	5 days	26/10/26 08:00	30/10/26 17:00	Thuận	Week 8	
29	Create Test Cases (Iteration 2)	5 days	28/10/26 08:00	03/11/26 17:00	Thi	Week 8-9	
30	Code, UT, IT (Iteration 2)	8 days	28/10/26 08:00	06/11/26 17:00	Thi	Week 8-9	
31	Complete Software Package 3	0 days	06/11/26 17:00	06/11/26 17:00	Whole team	Week 9	
32	Code Iteration 3 (Packages 4 & 5: Fulfillment & Analytics)	10 days	09/11/26 08:00	20/11/26 17:00		Week 10-11	Finalize the mobile app & reports
33	Update SRS & SDS (Iteration 3)	5 days	09/11/26 08:00	13/11/26 17:00	Thuận	Week 10	
34	Create Test Cases (Iteration 3)	5 days	11/11/26 08:00	17/11/26 17:00	Việt, Phượng	Week 10-11	
35	Code, UT, IT (Iteration 3)	8 days	11/11/26 08:00	20/11/26 17:00	Việt, Phượng	Week 10-11	
36	Complete Software Package 4 & 5	0 days	20/11/26 17:00	20/11/26 17:00	Whole team	Week 11	
37	Deliver Report 5 (Test Report)	0 days	20/11/26 17:00	20/11/26 17:00	Thuận	Week 11	
38	Verification (Stage 5)	10 days	23/11/26 08:00	04/12/26 17:00		Week 12-13	
39	Execute ST & Fix Bugs	10 days	23/11/26	04/12/26	Whole	Week	End-to-end testing of

#	Deliverable	Duration	Start	Finish	Resource Names	Week	Notes
			08:00	17:00	team	12-13	the whole system
40	Prepare for Deployment	10 days	23/11/26 08:00	04/12/26 17:00	Thi	Week 12-13	
41	Update Project Documents	10 days	23/11/26 08:00	04/12/26 17:00	Thuận	Week 12-13	
42	Complete Full Software Package	0 days	04/12/26 17:00	04/12/26 17:00	Whole team	Week 13	Complete MVP version
43	Transition (Stage 6)	10 days	07/12/26 08:00	18/12/26 17:00		Week 14-15	
44	Software Optimization	5 days	07/12/26 08:00	11/12/26 17:00		Week 14	
45	Optimize / Refactor Codes	3 days	07/12/26 08:00	09/12/26 17:00	Thi, Việt	Week 14	
46	Execute UAT & Fix Bugs	3 days	09/12/26 08:00	11/12/26 17:00	Whole team	Week 14	
47	Deploy Final Code	0 days	11/12/26 17:00	11/12/26 17:00	Thi	Week 14	
48	User Guides	5 days	07/12/26 08:00	11/12/26 17:00		Week 14	
49	Prepare User Guides	5 days	07/12/26 08:00	11/12/26 17:00	Phượng	Week 14	
50	Deliver Report 6 (User Guides)	0 days	11/12/26 17:00	11/12/26 17:00	Phượng	Week 14	
51	Prepare Final Project Report	4 days	14/12/26 08:00	17/12/26 17:00	Whole team	Week 15	
52	Prepare Thesis Presentation	1 day	18/12/26 08:00	18/12/26 17:00	Whole team	Week 15	Prepare defense presentation slides
53	Deliver Report 7 (Final Project Report)	0 days	18/12/26 17:00	18/12/26 17:00	Thuận	Week 15	

4. Responsibility Assignments

D~Do; R~Review; S~Support; I~Informed; <blank>-Omitted

Responsibility	Thi (Backend)	Thuận (Leader)	Việt (Frontend)	Phượng (Frontend)
Project Planning & Tracking	S	D	R	R
Prepare Project Introduction Document	S	S	D	R
Prepare SRS Document (Overview Part)	R	D	S	S
Prepare SRS Document (User Requirements)	D	R	S	S
Prepare Report 2 (Project Management Plan)	S	D	R	R
Write SDS (System Design Specification)	R	D	S	S
Code Package 1 & 2 (Core, Omnichannel Integration)	S	D	R	S
Code Package 3 (Packing Optimization)	R	D	S	I
Code Package 4 & 5 (Fulfillment, Analytics)	S	S	D	D
Unit / Integration / System Testing	R	D	D	S
Prepare for Deployment	D	D	S	S
Prepare User Guides	D	S	S	S
Prepare Final Project Report & Thesis Presentation	D	S	S	S

5. Project Communications

Communication Item	Who / Target	Purpose	When, Frequency	Type, Tool, Method(s)
Daily Scrum Meeting	Project team (all 4 members)	Daily progress update and blockers	Daily (15 minutes in the morning)	Online meeting (Google Meet) or in-person stand-up at school
Sprint Planning & Retrospective	Project team	Plan the new Sprint and review lessons learned	Start and end of each Sprint (every 2 weeks)	In person at the lab / Google Meet; notes recorded in Jira
Project Review	Project team &	Report progress, demo	Weekly, or as	In person at school

Communication Item	Who / Target	Purpose	When, Frequency	Type, Tool, Method(s)
(Mentoring)	supervisor	features and get guidance	scheduled by the supervisor	
Ad-hoc Discussions	Relevant members (e.g. back end with front end)	Resolve technical issues (API integration bugs, JSON structure changes)	As needed	Group chat on Zalo / Discord
Task & Bug Tracking Updates	Project team	Notification when a task status changes or a bug is logged	Continuous	Jira Kanban board

6. Configuration Management

6.1 Document Management

- Shared workspace: drafts and meeting notes are managed on Confluence; integration guides, API lists, design notes and diagram exports (draw.io) are versioned in the GitHub repository (folders GUIDE_DOC and docs).
- Final deliverables: official documents submitted to the school (Reports 1–7, SRS, SDD) are exported as Word / PDF files and stored on the team's shared Google Drive and in the repository for Review 2.
- Naming convention: officially submitted files follow the school template for each review (e.g. SWP490_FA26SE036_R2_Report2.pdf for Review 2); working drafts follow [Document Name]_[Group Code]_v[Version].docx.

6.2 Source Code Management

Source code is managed on GitHub (monorepo with npm workspaces: be, fe, storefront, plus the packer service):

- Branches: main holds the integrated, release-ready code; each member works on a personal branch (thi_dev, feature/thuan_be, feature/viet_ui, feature/phuong_admin, feature/phuong_ui) and short-lived topic branches for larger changes (e.g. feat/stock-lots-and-sync-settings, merge/… branches for integration).
- Merge process: changes reach main through Pull Requests. A PR is merged after review by at least one other member, a passing CI run and no conflicts.
- Commit message: Conventional Commits with the Jira ticket as scope, enforced by commitlint and a Husky commit-msg hook, header ≤ 100 characters, e.g. feat(AOFP-12): add lazada oauth connect flow. A Husky pre-commit hook runs the test suite.
- Continuous integration: GitHub Actions ci.yml runs on every push and on pull requests to main — back end (type check, lint, unit tests), staff portal (type check, build, lint), AURELLE web store (type check, build), packer (ruff, mypy, pytest), Docker image builds and Kubernetes manifest validation. release.yml builds and pushes Docker images to GitHub Container Registry on push to main, on v* tags or manually.

6.3 Tools & Infrastructures

Category	Tools / Infrastructure
Technology	NestJS 11 (back end, Node.js / TypeScript), React 19 + Vite (staff portal), Next.js 15 (AURELLE web store), Python 3.12 + FastAPI + OR-Tools CP-SAT (packer service), Flutter (mobile app, planned Iteration 3)
Database	MongoDB Atlas (primary database, Mongoose ODM); Redis (authentication-state cache)
IDEs / Editors	Visual Studio Code (front end & back end), Android Studio (mobile, Iteration 3)
Diagramming	draw.io (architecture, use case, ERD, activity and state diagrams, docs/diagrams), Figma (UI/UX design)
Documentation	Confluence, Microsoft Word, Google Docs / Sheets, Markdown guides in the repository
Version Control	GitHub (source code, pull requests, GitHub Actions CI)
Deployment	Docker Compose (be, fe, storefront, packer, redis), Kubernetes manifests (k8s/base with local and ghcr overlays), GitHub Container Registry, MongoDB Atlas. A hosted staging environment is not yet set up (risk R08).
Project management & API testing	Jira (Sprint / task / defect tracking), Swagger (OpenAPI docs at /api/docs), Postman